```
  override def addCargo(in: Cargo): Future[AddedCargo] = {
    Future.successful(AddedCargo(true))                                    ❷
  }
}
```

❶ Execution context with a thread pool to run the future

❷ Future is precomputed; returns immediately

Once this is implemented, you can create an instance of it. To pass the execution context, use the one from the `ActorSystem`. You can also add bidirectionality—reflection—to this service so you can call it from the command line and receive a response:

```
object ContainerServer {
  ...
  val service: HttpRequest => Future[HttpResponse] =
       ContainerServiceHandler
       .withServerReflection(new ContainerServiceImpl())    ❶
  ...
}
```

❶ Adds reflection with ContainerServiceHandler, which is created by akka-gRPC

Adding reflection means a consumer can send a text to the server and have it automatically converted to the protobuf format the server understands. This also works with the response being converted to text, which is what humans understand. Otherwise, the consumer must send the proto definitions for the request and the response along with the message.

Once you have the service instance, you can bind it to `Http().newServerAt()`, as you've seen before. The entire application with the gRPC server is shown next.

Listing 11.7 gRPC server

```
import akka.actor.typed.ActorSystem
import akka.actor.typed.scaladsl.Behaviors
```